[RFC] Enhancement of parallel processing capability for primary key tables

Background

User Feedback

1. In a data skew scenario at Intuit, a single tablet processes half of the data writes. e.g. Intuit Maichimp Ingestion issue
2. In a scenario where too few buckets are configured—for example, in Weave—they want to insert 120GB data into primary key table with only 3 buckets, which resulted in excessively high publish latency for the primary key table.
3. One of our community user reported that during real-time ingestion, latency is generally normal most of the time. However, once a full compaction occurs, the publish operation takes much longer, causing severe ingestion latency jitter.

   From the trace, we can see that most of the time is spent on upserts in the primary key index memtable and on generating SST files. The index memtable was filled 12 times, resulting in 12 SST files being generated.


```
{
    "child_traces": [
        [
            "PublishTablet",
            {
                "compaction_delvec_loader_latency_us": 90,
                "compaction_get_next_latency_us": 509312,
                "compaction_replace_index_latency_us": 36328846,
                "input_rowsets_size": 4,
                "max_rowsetid": 7751,
                "minor_compact_latency_us": 11671226,
                "minor_compact_times": 12,
                "output_rowsets_size": 3,
                "pindex_memtable_replace_us": 22255983,
                "primary_index_commit_latency_us": 11,
                "primary_index_load_latency_us": 0,
                "queuing_latency_us": 17,
                "tablet_id": 1291314
            }
        ]
    ]
}
```


Summary

These cases scenario a common characteristic:

a. A single bucket is too large because of data skew or unreasonable bucket configuration.

b. Heavy imports or full compactions.

And it will cause the primary key index in a single tablet to undergo massive modifications, and because the publishing execution within a single tablet cannot run in parallel, so it will cause a bottleneck in the overall import latency.

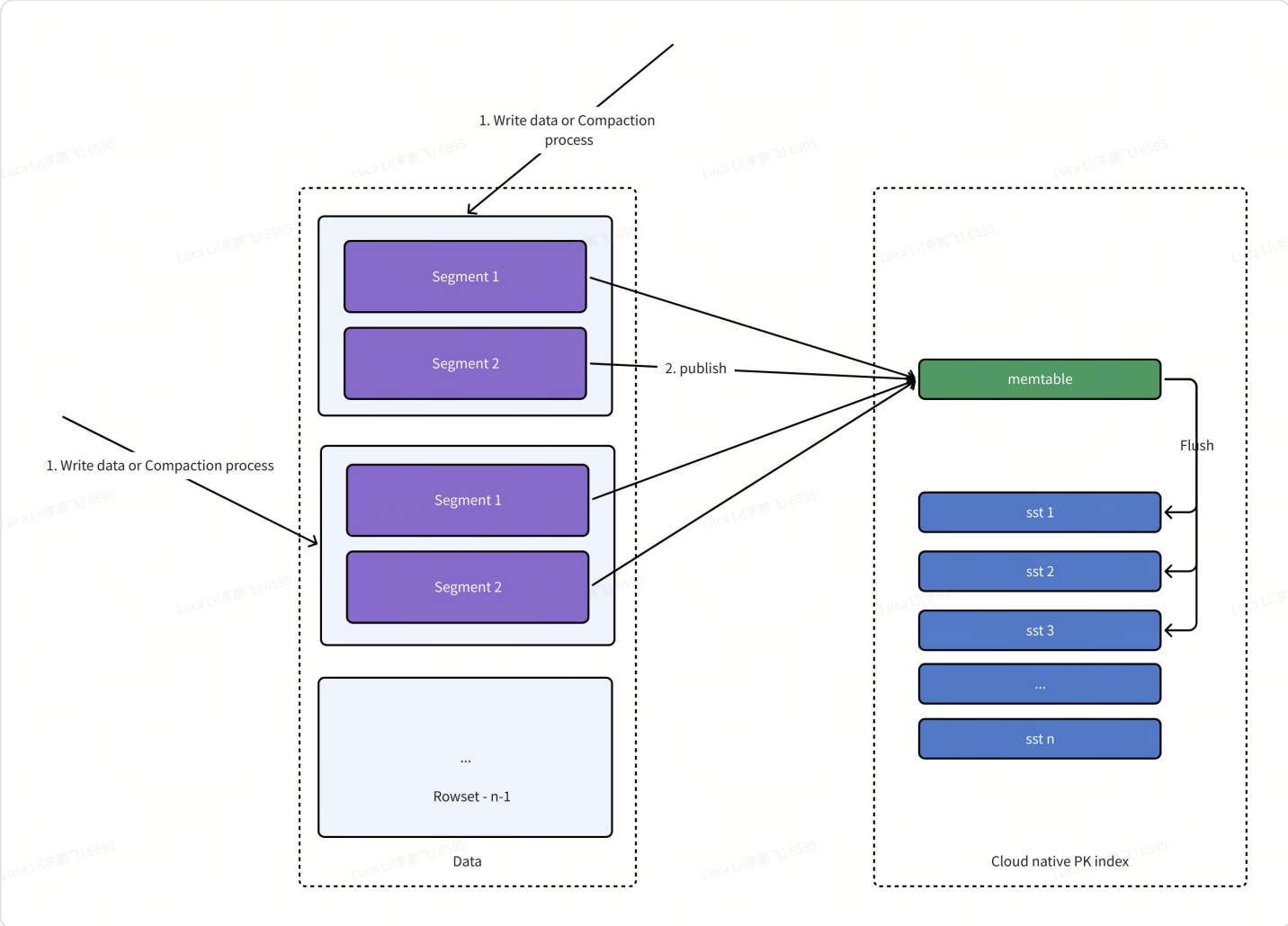
Goal

The processing capacity of the primary key table is no longer limited by the number of tablets. Even if the import involves only a single tablet, or the data volume within a single tablet is very large, parallel processing within the tablet can still be used to reduce latency.

The parallel execution here includes two parts:

1. **Primary key index files generation** is parallelized **across transactions**.
2. **Primary key index lookups** are executed in parallel **within a single transaction**.

Current Process Review



1. During the data import and compaction execution phases, StarRocks will generate rowsets with segment files, and for the same tablet, the data import and compaction phase can be executed in parallel.
2. During the publishing phase, different transactions on the same tablet must be executed serially. At this stage, PK index updates are involved, and if the memtable is full, several SST files will be generated.
3. If it is an import transaction, the index also needs to be read to locate the physical positions of the updated rows and generate the delete vector. In contrast, if it is a compaction transaction, it only needs to read the historical delvec to complete the generation of the new delete vector.

> We don't need to read primary key index during compaction transaction publishing because of this : [RFC] PK表compaction事务publish & apply执行优化方案

Based on the above process, we can see that the time-consuming operations in the publish phase (such as modifying and reading the primary key index) directly affect the parallel execution efficiency of a single tablet.

Design

Hybrid Global & Local Index

Some industry products actually adopt a local index approach. Their primary key index is stored together with the segment files, and the index can be built during the data ingestion phase.

StarRocks has adopted a global index approach for primary key indexing from the very beginning.

Here is the comparsion:

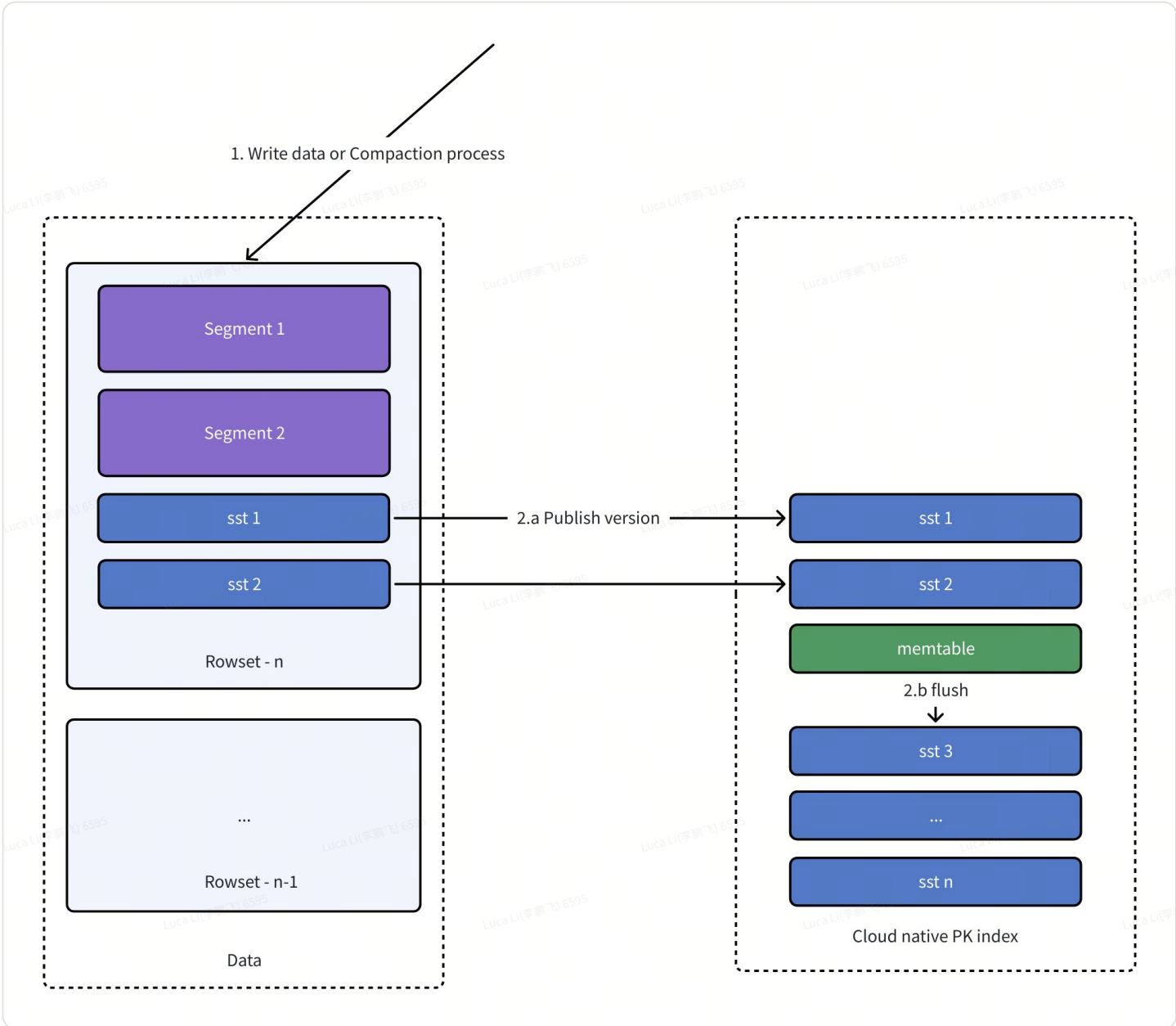
Comparison Dimension	Local Index Approach	Global Index Approach
Index Construction Timing	Can be updated during the import phase; maintained within each segment	Can only be updated at the publish (because global rowsetid + segmer must be determined)
Real-Time Import Performance	Stable import latency, but can generate a large number of small index files	Uses memtable aggregation to red small files, more efficient in real-tir scenarios
Number of Index Files	Each segment has its own index, resulting in more files	Supports independent index comp fewer files generated
Publish Phase Duration	Relatively short; publish is not a bottleneck	Index updates are concentrated in publish phase, which may cause pu delays
Parallelism	Index updates can be performed in parallel per transaction; publish not heavily blocked	Index generation for the same tabl serialized; large tablets may form a bottleneck
Memory Usage	Does not rely on large memtables; more stable	Relies on memtable; in large impor compaction, many index files may generated
Storage Format	Standard storage format; point lookup efficiency is average	Can use point-lookup-friendly stora format; better query performance
Suitable Scenarios	Suitable for high-throughput scenarios like bulk import and full compaction, where many index files are tolerable	Suitable for real-time high-frequen imports

Therefore, in this document, I propose a hybrid strategy with the following characteristics:

1. SSTable files and segment files remain independently stored, ensuring that SSTable files can undergo compaction independently, without sacrificing the advantages of the current global index.
2. During the data import phase (compaction execution phase), the system identifies whether it is a large import or compaction scenario. If so, SSTables are pre-generated. At this stage, the SSTables contain only local rowset IDs and do not include global rowset IDs + segment IDs. Even for the same tablet, this process can be executed in parallel during the import phase.
3. During the publish phase, the pre-generated SSTable files are incorporated into the primary key index metadata, and the global rowset IDs + segment IDs are set in `PersistentIndexSstablePB`.

The current design can leverage the advantages of both global and local index approaches, and achieve excellent parallel processing capability.

Process flow

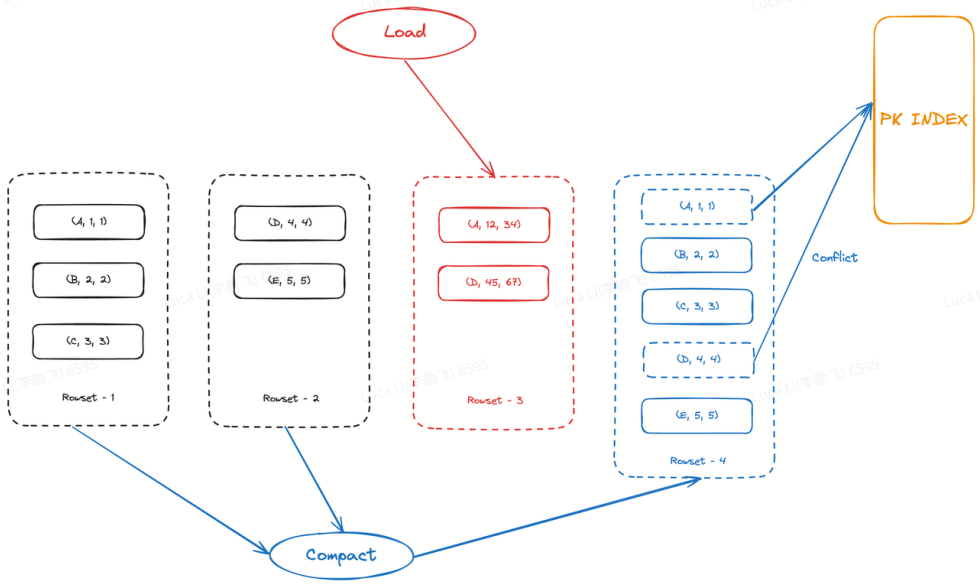


1. During the data import and compaction execution phases, SSTables corresponding one-to-one with segment files are generated. However, since the rowset ID of the newly generated rowset is not yet known, the values in these pre-generated SSTables can only contain the row IDs at this stage.
2. During the publishing phase:
   a. The SSTables generated during the import and compaction execution phases are directly added to the cloud-native index' s SSTable list, and the final determined RSSID is recorded in `PersistentIndexSstablePB`.
   b. If there are any unflushed memtables at this time, they need to be flushed into SSTables beforehand.

c. If it is an import transaction, the index also needs to be read to locate the physical positions of the updated rows and generate the delete vector. In contrast, if it is a compaction transaction, it only needs to read the historical delvec to complete the generation of the new delete vector.

Handling Conflicts Between Compaction and Import

During execution, compaction may conflict with ongoing imports, as these conflicts need to be resolved during the publish phase of compaction. For example:



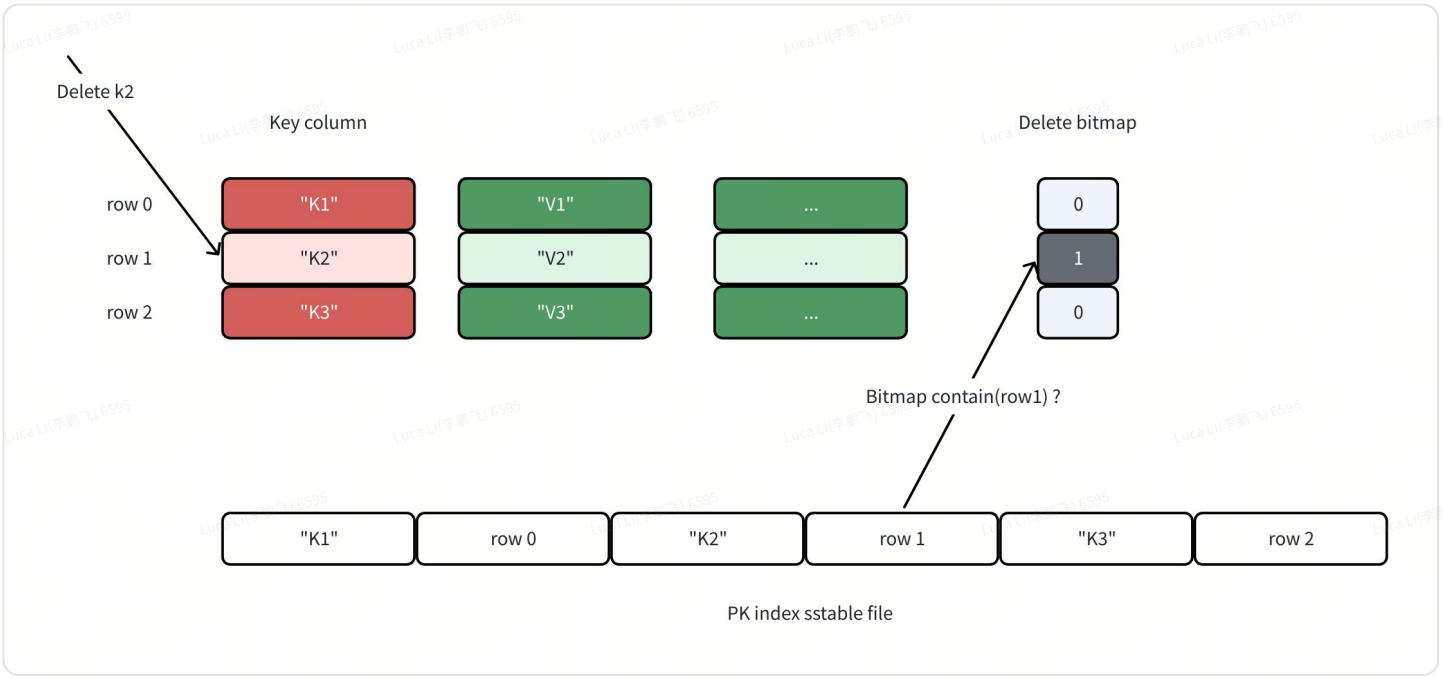
In the figure above, during the compaction of `rowset-1` and `rowset-2`, `rowset-3` generated by an import is inserted. It modifies rows A and D, so in the final compaction-generated `rowset-4`, rows A and D need to be marked as deleted.

During the compaction publish phase, we determine whether a row generated by the compaction should be marked as deleted by querying the latest delete vector (delvec) of the merged segments.

For SST files generated during the compaction phase, if a row in the data file has been marked as deleted, the SST file itself also needs to reflect this deletion.

We can introduce a flag in `PersistentIndexSstablePB` to share a delete vector bitmap with the segment files.

When reading an SST, the corresponding deletion bitmap is loaded, and using the bitmap's `contains` check, the deleted key-value pairs can be filtered out.



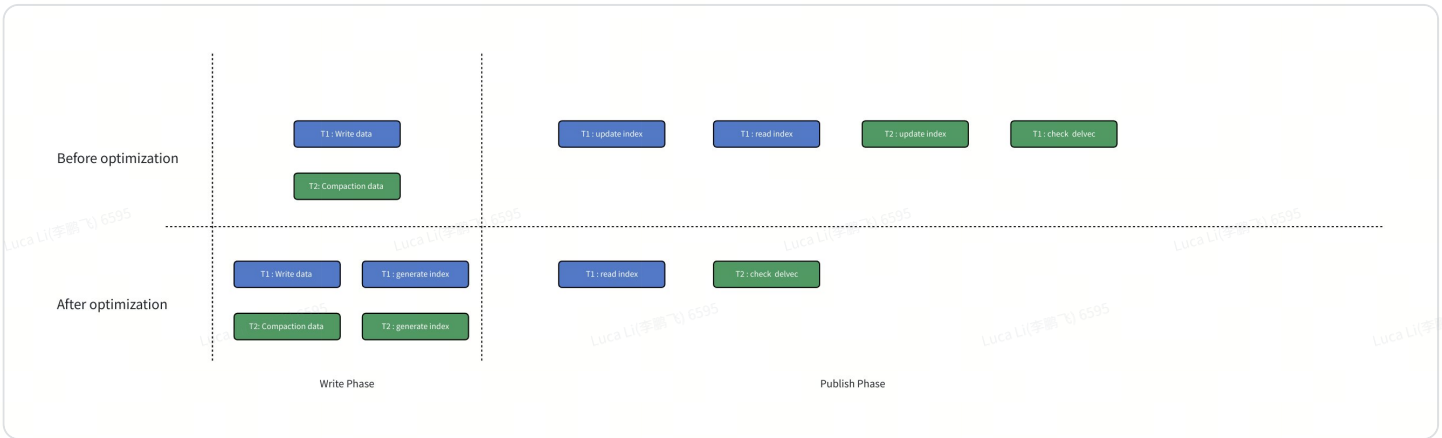
Publish Time Analysis

Before Optimization

1. Data import: Update primary key index + Read primary key index
2. Compaction: Check `delvec` + Update primary key index

After Optimization

2. Data import: Read primary key index
3. Compaction: Check `delvec`

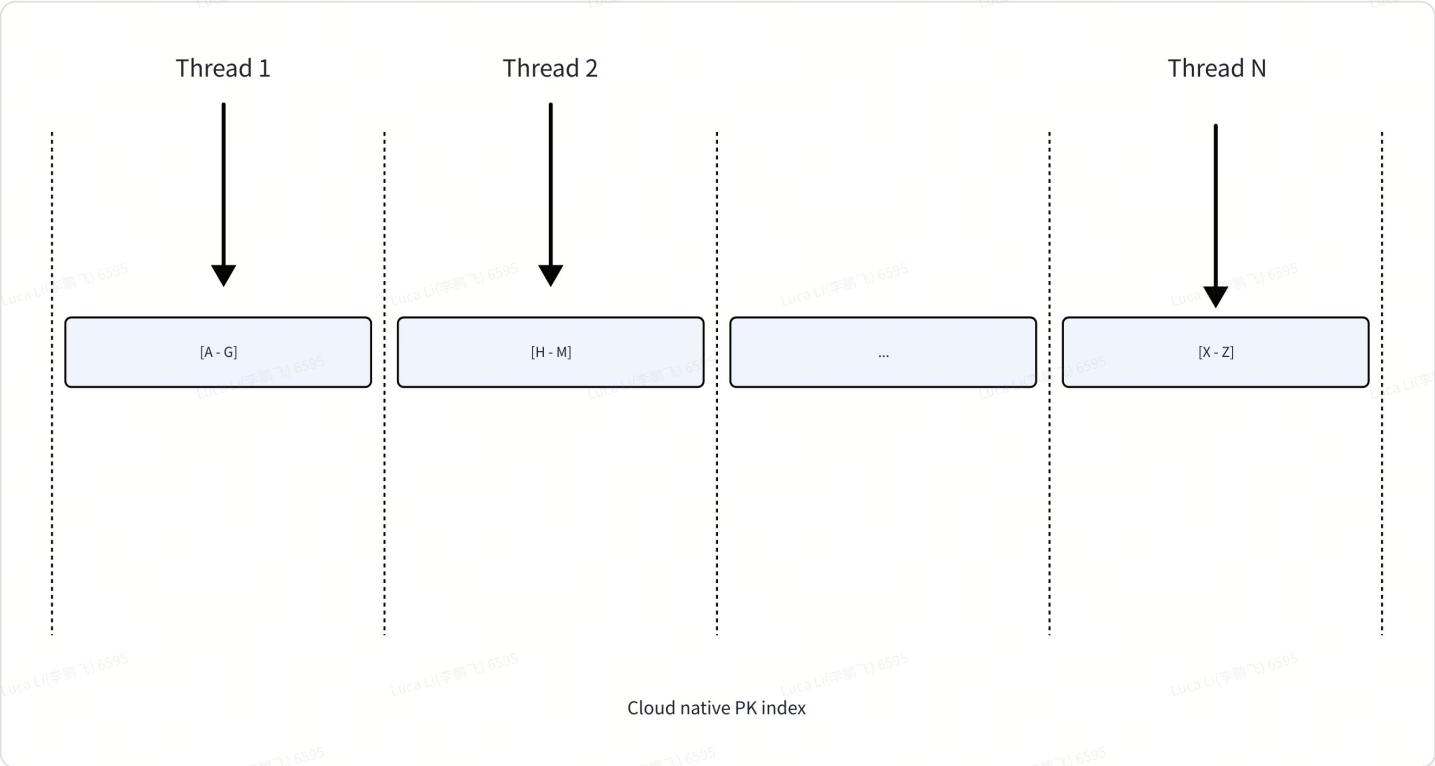


Since the publish phase is the conflict resolution stage, all import and compaction transactions for the same tablet must be executed serially during publish. Therefore, by moving the

generation of index files to the import/compaction phase, this solution can improve the overall parallelism of imports.

Parallel PK Index Lookup within a Single Tablet

After completing the above optimizations, during the serial publish phase, the only remaining time-consuming operation is the primary key index lookup. Since the cloud-native index is implemented in an ordered format, further parallelization can be applied here by splitting the scan range into multiple chunks and executing them concurrently across multiple threads.



Temporarily Unsupported Scenarios

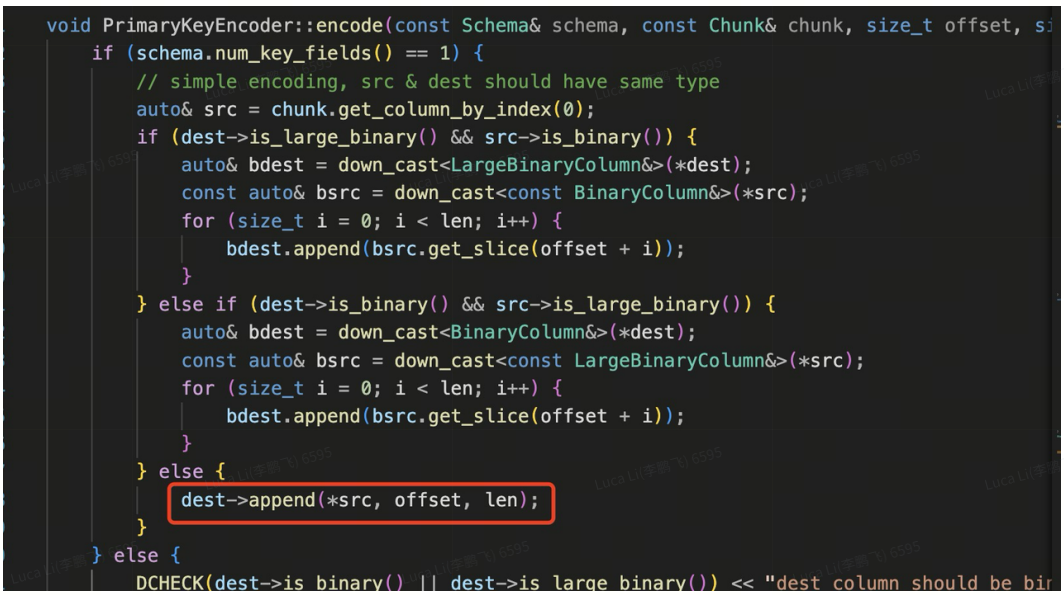
1. Sort key and primary key orders are inconsistent:
   When the sort key order differs from the primary key order, an additional sorting phase is required during the import phase to generate ordered SSTables. This would significantly increase import overhead, so it will not be implemented in the first phase.
2. Primary key consists of a single non-VARCHAR/CHAR column:
   This optimization cannot be applied. The reason is that, in the current primary key model, when encoding a single-key column of a non-binary type, big-endian encoding is not used, which may result in:

代码块

```
253-> FD000000
```

```
254-> FE000000
255-> FF000000
256-> 00010000 -> unorder after encode
257-> 01010000
```

This is a legacy bug, but for compatibility reasons, it will not be supported in the first phase.



Verification Results via POC

How

I quickly created a beta version that supports these features, to verify that our proposed design provides tangible benefits:

1. Generating SSTables during the import & compaction phase
2. Parallel PK Index Lookup within a Single Tablet. (100MB per unit)

Test Cluster Size

Instance id	Instance ip	Instance type	Storage size(G)	Disk count	Disk type	Instance status
i-8vb93s2k6n44m5yugmy1	172.26.80.35	ecs.u1-c1m2.xlarge	40	1	cloud_essd	Running
i-8vbg58ldr5l40hvryzr5	172.26.80.33	ecs.u1-c1m2.4xlarge	256	1	cloud_essd	Running

Test 1 - Large Import Test

Create table

Only use one bucket.

代码块

```
CREATE TABLE contact_merge_fields_v4_query_mv (
               account_id varchar(256) NOT NULL COMMENT "",
               contact_id varchar(256) NOT NULL COMMENT "",
               merge_field_id varchar(256) NOT NULL COMMENT "",
               account_provider varchar(256) NULL COMMENT "",
               contact_audience_id largeint(40) NULL COMMENT "",
               merge_field_value_date datetime NULL COMMENT "",
               merge_field_value_numeric float NULL COMMENT "",
               merge_field_value_string varchar(256) NULL COMMENT "",
               merge_field_display_name varchar(256) NULL COMMENT "",
               merge_field_key varchar(256) NULL COMMENT ""
             ) ENGINE=OLAP
             PRIMARY KEY(account_id, contact_id, merge_field_id)
             DISTRIBUTED BY HASH(account_id) BUCKETS 1
             ORDER BY(account_id, contact_id, merge_field_id)
             PROPERTIES (
                "replication_num" = "1"
             )
```

Data size after two data imports:

代码块

```
mysql> show data;
+-----------------------------------+---------------+---------------------+
| TableName                         | Size          | ReplicaCount        |
+-----------------------------------+---------------+---------------------+
| contact_merge_fields_v4_query_mv | 7.078 GB      | 1                   |
| Total                             | 7.078 GB      | 1                   |
| Quota                             | 8388608.000 TB | 9223372036854775807 |
| Left                              | 8388607.993 TB | 9223372036854775806 |
+-----------------------------------+---------------+---------------------+

mysql> select count(*) from contact_merge_fields_v4_query_mv;
+-----------+
| count(*)  |
+-----------+
| 117353566 |
+-----------+
```

First data import

Imported a CSV file with 58,676,783 rows, with a file size of 6.5 GB.

	Write data cost	Publish cost	Total cost
Before	281017 ms	279610 ms	560627 ms
After	368436 ms	20925 ms	389361 ms
Compare	+31%	-92.5 %	-31%

Publish latency is 13.3x faster than before!!!

Before optimization

代码块

```
I20250825 16:26:28.825160 22771337717312 lake_service.cpp:316] Published
txns=. tablets=10586 cost=279398280us, trace: {"child_traces":
[["PublishTablet",
{"base_version":1,"continue_block_read_cnt":0,"deletes":0,"do_update_latency_us
":269310783,"load_segment_us":10076640,"minor_compact_latency_us":87186219,"min
or_compact_times":15,"multi_get_us":123670231,"multiget_t1_us":8919284,"multige
t_t2_us":56657076,"multiget_t3_us":38652818,"new_del":0,"pindex_memtable_upsert
_us":25223024,"primary_index_commit_latency_us":18,"primary_index_load_latency_
us":71,"queuing_latency_us":53,"read_block_hit_cache_cnt":2608727,"read_block_m
iss_cache_cnt":490641,"rewrite_segment_latency_us":150,"rowsetid":1,"sst_bloom_
filter_rows":308508969,"tablet_id":10586,"total_del":0,"update_index_latency_us
":269314036,"upsert_rows":58676783,"upserts":5}]]}
```

It can be observed that 15 minor compactions were triggered. The time to scan the SSTables also reached as high as 123 seconds.

After optimization

代码块

```
I20250828 16:38:45.029261 22444039296576 lake_service.cpp:316] Published
txns=. tablets=11270 cost=20891048us, trace: {"child_traces":[["PublishTablet",
{"base_version":1,"deletes":0,"do_update_latency_us":19530575,"load_segment_us"
:1186517,"new_del":0,"primary_index_commit_latency_us":9,"primary_index_load_la
tency_us":71,"queuing_latency_us":30,"rewrite_segment_latency_us":148,"rowsetid
```

":1,"tablet_id":11270,"total_del":0,"update_index_latency_us":19700568,"upsert_rows":58676783,"upserts":5}]]}

Second data import

Imported another CSV file with 58,676,783 rows, with a file size of 6.5 GB.

	Write data cost	Publish cost	Total cost
Before	279461 ms	589383 ms	868844 ms
After	358891 ms	43081 ms	401972 ms
Compare	+28%	-93 %	-54 %

Publish latency is 13.6x faster than before!!!

Before optimization

代码块

```
I20250825 16:47:29.852787 22771352426048 lake_service.cpp:316] Published txns=. tablets=10586 cost=588478229us, trace: {"child_traces":[["PublishTablet",
{"base_version":2,"continue_block_read_cnt":0,"deletes":0,"do_update_latency_us":578522642,"load_segment_us":8528308,"minor_compact_latency_us":85118325,"minor_compact_times":16,"multi_get_us":404472017,"multiget_t1_us":29345555,"multiget_t2_us":178509034,"multiget_t3_us":132649729,"new_del":0,"pindex_memtable_upsert_us":25778081,"primary_index_commit_latency_us":36,"primary_index_load_latency_us":1421688,"queuing_latency_us":31,"read_block_hit_cache_cnt":7252926,"read_block_miss_cache_cnt":2714979,"rebuild_index_segment_cnt":1,"rebuild_index_segment_cost_us":1102329,"rewrite_segment_latency_us":148,"rowsetid":6,"sst_bloom_filter_rows":1026390562,"tablet_id":10586,"total_del":0,"total_num_rows":58676783,"total_segment_cnt":5,"update_index_latency_us":578527594,"upsert_rows":58676783,"upserts":5}]]}
```

After optimization

代码块

```
I20250828 16:51:41.509084 22443862513216 lake_service.cpp:316] Published txns=. tablets=11270 cost=43051980us, trace: {"child_traces":[["PublishTablet",
```

{"base_version":2,"deletes":0,"do_update_latency_us":39521128,"load_segment_us":1789952,"new_del":0,"primary_index_commit_latency_us":15,"primary_index_load_latency_us":1541262,"queuing_latency_us":23,"rebuild_index_segment_cnt":1,"rebuild_index_segment_cost_us":1383746,"rewrite_segment_latency_us":138,"rowsetid":6,"tablet_id":11270,"total_del":0,"total_num_rows":58676783,"total_segment_cnt":5,"update_index_latency_us":39720205,"upsert_rows":58676783,"upserts":5}]]}

Parallel data import

We initiated two import tasks simultaneously and measured the total elapsed time from the start until the completion of the last import task. Every task writes CSV file with 58,676,783 rows, with a file size of 6.5 GB.

	Duration	Total cost
Before	11:08:27 -> 11:43:10	34 min 43 s (2083000 ms)
After	10:43:18 -> 10:51:12	7 min 54 s (474000 ms)
Compare		4.4x faster

After optimization

代码块

```
1  2025-08-29 10:50:28.894+08:00 INFO (lake-publish-task-29|273)
   [DatabaseTransactionMgr.finishTransaction():1340] finish transaction
   TransactionState. txn_id: 73, label: contact_merge_fields_load_1d1c1f24, db
   id: 10228, table id list: 10241, callback id: [10245], coordinator: BE:
   172.26.80.9, transaction status: VISIBLE, error replicas num: 0, unknown
   replicas num: 0, prepare time: 1756435398762, write end time: 1756435808500,
   allow commit time: 1756435808500, commit time: 1756435808500, finish time:
   1756435828890, write cost: 409738ms, publish total cost: 20390ms, total cost:
   430128ms, reason: , attachment:
   com.starrocks.load.loadv2.ManualLoadTxnCommitAttachment@7b679ca6, partition
   commit info:[partitionId=10243, version=2, versionTime=1756435828890,
   isDoubleWrite=false,], warehouse: 0 successfully
2  2025-08-29 10:51:12.503+08:00 INFO (lake-publish-task-30|275)
   [DatabaseTransactionMgr.finishTransaction():1340] finish transaction
   TransactionState. txn_id: 74, label: contact_merge_fields_load_4c8eea61, db
```

```
id: 10228, table id list: 10241, callback id: [10246], coordinator: BE:
172.26.80.9, transaction status: VISIBLE, error replicas num: 0, unknown
replicas num: 0, prepare time: 1756435402939, write end time: 1756435810415,
allow commit time: 1756435810415, commit time: 1756435810415, finish time:
1756435872501, write cost: 407476ms, publish total cost: 62086ms, total cost:
469562ms, reason: , attachment:
com.starrocks.load.loadv2.ManualLoadTxnCommitAttachment@145110a0, partition
commit info:[partitionId=10243, version=3, versionTime=1756435872501,
isDoubleWrite=false,], warehouse: 0 successfully
```

Test 2 - Micro-batch Import Test

Create table

Only use one bucket.

代码块

```
CREATE TABLE IF NOT EXISTS test_pri_load.tbl_pk_withvarchar (
  `lo_orderkey` string,
  `lo_linenumber` int(11),
  `lo_custkey` int(11),
  `lo_partkey` int(11),
  `lo_suppkey` int(11),
  `lo_orderdate` varchar(15),
  `lo_orderpriority` varchar(16) default "hello world",
  `lo_shippriority` int(11),
  `lo_quantity` int(11),
  `lo_extendedprice` int(11),
  `lo_ordtotalprice` int(11),
  `lo_discount` int(11),
  `lo_revenue` int(11),
  `lo_supplycost` int(11),
  `lo_tax` int(11),
  `lo_commitdate` bigint(11),
  `lo_shipmode` varchar(11)
)
PRIMARY
KEY(lo_orderkey,lo_linenumber,lo_custkey,lo_partkey,lo_suppkey,lo_orderdate)
COMMENT "OLAP"
DISTRIBUTED BY HASH(lo_orderkey) BUCKETS 1
PROPERTIES (
"replication_num" = "1",
"enable_persistent_index" = "true",
"replicated_storage" = "true"
```

```
);
```

Data import

Import 100 times consecutively, each time importing an 80MB CSV file.

代码块

```
for loop_file in $(seq 0 100)
  do
    echo `date`
    sleep 2
    uuid=$(uuidgen |sed 's/-//g')
    curl --location-trusted -u root: -T
/home/disk3/luoyixin/tools/bench_tool/data_2.5g/fake_data$loop_file.csv -XPUT -
H label:stream_load_$uuid -H "timeout:86400" -H "max_filter_ratio:0.1"
http://$host_ip:$http_port/api/$database_name/$table_name/_stream_load
  done
```

Total rows

代码块

```
mysql> select count(*) from tbl_pk_withvarchar;
+----------+
| count(*) |
+----------+
| 46202900 |
+----------+
1 row in set (0.09 sec)
```

	Write data cost	Publish cost	Total cost
Before	194601 ms (min/max : 1710/2634)	278047 ms (min/max : 300/11605)	472648 ms
After (force generate sst when loading)	242508 ms (min/max : 2224/3043)	246802 ms (min/ max : 491/4105)	489310 ms
Compare	+24%	-12%	+3%
After (adaptive)	205638 ms	206603 ms	412241 ms

	(min/max : 1851/3773)	(min/max : 363/3834)	
Compare	+5%	-25%	-13%

Plot the publish times of these 100 iterations as a chart:

图表 [该内容不支持导出查看]

It can be seen that the publish time after optimization is very stable, increasing slowly as the tablet grows. However, before optimization, the publish time was heavily affected by compaction publish, resulting in significant fluctuations.

Analysis

- **Before optimize**

If the data import transaction's publish does not encounter a memtable flush, the time cost is around 2 seconds, with the main overhead coming from `multi_get_us`, which involves reading the index files.

代码块

```
I20250827 16:41:48.331447 22669075736128 lake_service.cpp:316] Published
txns=. tablets=10635 cost=2132873us, trace: {"child_traces":[["PublishTablet",
{"base_version":107,"continue_block_read_cnt":0,"deletes":0,"do_update_latency_
us":2049596,"load_segment_us":74155,"multi_get_us":1660712,"multiget_t1_us":104
082,"multiget_t2_us":747042,"multiget_t3_us":581939,"new_del":0,"pindex_memtabl
e_upsert_us":231574,"primary_index_commit_latency_us":12,"primary_index_load_la
tency_us":0,"queuing_latency_us":26,"read_block_hit_cache_cnt":26345,"read_bloc
k_miss_cache_cnt":10932,"rewrite_segment_latency_us":28,"rowsetid":107,"sst_blo
om_filter_rows":3950819,"tablet_id":10635,"total_del":0,"update_index_latency_u
s":2053190,"upsert_rows":462029,"upserts":1}]]}
```

If the memtable happens to be full, it will trigger a memtable flush, causing the time cost to rise to around 5 seconds.

代码块

```
I20250827 16:40:16.840386 22668928128576 lake_service.cpp:316] Published
txns=. tablets=10635 cost=4944397us, trace: {"child_traces":[["PublishTablet",
{"base_version":94,"continue_block_read_cnt":0,"deletes":0,"do_update_latency_u
s":4875959,"load_segment_us":63226,"minor_compact_latency_us":1525933,"minor_co
mpact_times":1,"multi_get_us":2890883,"multiget_t1_us":97016,"multiget_t2_us":7
11190,"multiget_t3_us":1866449,"new_del":0,"pindex_memtable_upsert_us":238456,"
primary_index_commit_latency_us":12,"primary_index_load_latency_us":0,"queuing_
```

```
latency_us":27,"read_block_hit_cache_cnt":25721,"read_block_miss_cache_cnt":883
7,"rewrite_segment_latency_us":27,"rowsetid":94,"sst_bloom_filter_rows":3697139
,"tablet_id":10635,"total_del":0,"update_index_latency_us":4876035,"upsert_rows
":462029,"upserts":1}]]}
```

However, if a compaction occurs, the publish time can spike to 9 seconds.

代码块

```
1  I20250827 16:41:31.961770 22669075736128 lake_service.cpp:316] Published
   txns=. tablets=10635 cost=9019931us, trace: {"child_traces":[["PublishTablet",
   {"compaction_delvec_loader_latency_us":309,"compaction_get_next_latency_us":419
   065,"compaction_replace_index_latency_us":8326121,"input_rowsets_size":13,"max_
   rowsetid":92,"minor_compact_latency_us":5312854,"minor_compact_times":4,"output
   _rowsets_size":1,"pindex_memtable_replace_us":896790,"primary_index_commit_late
   ncy_us":11,"primary_index_load_latency_us":1,"queuing_latency_us":27,"tablet_id
   ":10635}]]}
```

- **After optimize**

After optimization, the publish time for import transactions is very stable, around 2 seconds, and increases slowly as the tablet grows, with the main overhead coming from reading the index files `multi_get_us` .

代码块

```
1  I20250828 11:40:44.805710 23081787418176 lake_service.cpp:316] Published
   txns=. tablets=10383 cost=2198343us, trace: {"child_traces":[["PublishTablet",
   {"base_version":103,"continue_block_read_cnt":0,"deletes":0,"do_update_latency_
   us":2112328,"load_segment_us":76292,"multi_get_us":1706750,"multiget_t1_us":107
   893,"multiget_t2_us":822278,"multiget_t3_us":544850,"new_del":0,"pindex_memtabl
   e_upsert_us":233135,"primary_index_commit_latency_us":13,"primary_index_load_la
   tency_us":0,"queuing_latency_us":27,"read_block_hit_cache_cnt":34351,"read_bloc
   k_miss_cache_cnt":9074,"rewrite_segment_latency_us":33,"rowsetid":103,"sst_bloo
   m_filter_rows":4576865,"tablet_id":10383,"total_del":0,"update_index_latency_us
   ":2116374,"upsert_rows":462029,"upserts":1}]]}
```

For compaction's publish, the time cost is very low and has no impact on imports.

- **Disable compaction**

If the publish latency jitter before optimization was caused by compaction transaction publishes, what would happen if I disabled compaction? I went ahead and performed this set of comparison tests:

	Publish cost
Before optimize	278047 ms (min/max : 300/11605)
Before optimize（disable compact）	357737 ms (min/max : 338/7290)

It can be seen that the jitter indeed disappeared, but as the number of index files gradually increases, the publish time also rises rapidly.

图表 [该内容不支持导出查看]

When to Enable Optimization

TBD